LA MINUTE DE CODE



FORMATION: JAVASCRIPT DE A À Z

PARTIE 2

LA MINUTE DE CODE

PLAN DE LA FORMATION

JS

- Les conditions avec If et else if
- Les conditions avec switch
- Système Parents | Enfants
- La boucle for
- Les boucles while et do while
- Les tableaux
- Les méthodes liées aux tableaux
- La boucle for of
- La méthode value

LA MINUTE DE CODE

CONDITION AVEC IF ET ELSE IF

Les conditions if et else if en JavaScript permettent de créer des branches de code qui sont exécutées en fonction d'une condition spécifiée. Les blocs de code if et else if peuvent être utilisés pour effectuer différentes actions en fonction de différentes valeurs.

Voici un exemple d'utilisation de if et else if :

```
let x = 5;

if (x > 10) {
  console.log("x est supérieur à 10");
} else if (x < 5) {
  console.log("x est inférieur à 5");
} else {
  console.log("x est compris entre 5 et 10");
}
```

Dans cet exemple, nous avons créé une variable x qui contient la valeur 5. Nous utilisons ensuite un bloc if pour vérifier si x est supérieur à 10, un bloc else if pour vérifier si x est inférieur à 5, et un bloc else pour gérer toutes les autres valeurs de x.

Si la condition spécifiée dans le bloc if est vraie, le code à l'intérieur du bloc est exécuté. Si elle est fausse, le code à l'intérieur du bloc else if est exécuté. Si toutes les conditions précédentes sont fausses, le code à l'intérieur du bloc else est exécuté.

CONDITION AVEC IF ET ELSE IF

Il est important de noter que si plusieurs blocs else if sont utilisés, ils sont évalués dans l'ordre dans lequel ils sont écrits. Le premier bloc else if dont la condition est vraie est celui qui sera exécuté.

Les conditions if et else if peuvent également être utilisées en conjonction avec des opérateurs de comparaison tels que ==, >, <, >=, <=, !=, !== pour créer des conditions plus complexes.

```
let age = 25;
let estMajeur = false;

if (age >= 18 && age <= 65) {
  estMajeur = true;
} else if (age < 18) {
  console.log("Vous êtes trop jeune pour
être majeur");
} else {
  console.log("Vous êtes trop âgé pour être
majeur");
}

if (estMajeur) {
  console.log("Vous êtes majeur");
} else {
  console.log("Vous n'êtes pas majeur");
}
```

CONDITION AVEC IF ET ELSE IF

Dans cet exemple, nous avons utilisé l'opérateur && pour créer une condition qui vérifie si l'âge est compris entre 18 et 65 ans. Si c'est le cas, la variable estMajeur est définie sur true.

Sinon, nous utilisons des blocs else if et else pour afficher des messages différents en fonction de l'âge.
Enfin, nous utilisons une seconde condition if pour vérifier la valeur de la variable estMajeur et afficher un message en conséquence.

LA MINUTE DE CODE

CONDITION AVEC SWITCH

JS

La condition switch en JavaScript permet de créer une branche de code qui est exécutée en fonction d'une valeur spécifiée. Cette structure de contrôle est souvent utilisée lorsque vous avez un grand nombre de conditions à vérifier.

Voici un exemple d'utilisation de switch :

```
let couleur = "bleu";

switch (couleur) {
  case "rouge":
    console.log("La couleur est rouge");
    break;
  case "bleu":
    console.log("La couleur est bleu");
    break;
  case "vert":
    console.log("La couleur est vert");
    break;
  default:
    console.log("La couleur n'est ni rouge,
ni bleu, ni vert");
}
```

LA MINUTE DE CODE

CONDITION AVEC SWITCH

Dans cet exemple, nous avons créé une variable couleur qui contient la valeur "bleu". Nous utilisons ensuite un bloc switch pour vérifier la valeur de la variable couleur. Nous avons utilisé plusieurs cas (case) pour vérifier si la valeur est égale à "rouge", "bleu" ou "vert". Si c'est le cas, le code à l'intérieur du bloc est exécuté. Si aucune des conditions précédentes n'est vraie, le code à l'intérieur du bloc default est exécuté.

Il est important de noter que chaque cas doit se terminer par un break. Cela permet d'éviter l'exécution de tous les blocs suivants même si leur condition est vraie.

LA MINUTE DE CODE

CONDITION AVEC SWITCH

JS

La condition switch peut également être utilisée avec des expressions plus complexes. Par exemple :

Dans cet exemple, nous utilisons la méthode getDay() de l'objet Date pour obtenir le jour de la semaine. Nous avons utilisé plusieurs cas pour vérifier si le jour est un jour de la semaine (du lundi au vendredi). Si c'est le cas, le message "C'est un jour de la semaine" est stocké dans la variable message. Si le jour est un dimanche ou un samedi, le message correspondant est stocké dans la variable. Si le jour n'est aucun de ceux-ci, un message d'erreur est stocké dans la variable.

```
let jour = new Date().getDay();
let message;

switch (jour) {
  case 0:
    message = "C'est dimanche";
    break;
  case 1:
  case 2:
  case 3:
  case 4:
  case 5:
    message = "C'est un jour de la semaine";
    break;
  case 6:
    message = "C'est samedi";
    break;
  default:
    message = "Le jour n'est pas valide";
}

console.log(message);
```

LA MINUTE DE CODE

CONDITION AVEC SWITCH

JS

La condition switch est utile lorsque vous avez de multiples conditions à vérifier et que vous voulez éviter d'utiliser plusieurs blocs if et else if.

Cependant, il est important de noter que la condition switch n'est pas toujours la meilleure option en termes de lisibilité et de maintenabilité du code. Il est donc important d'évaluer si son utilisation est pertinente pour votre cas d'utilisation spécifique.

LA MINUTE DE CODE

PARENT / ENFANTS

JS

En programmation web, l'arborescence DOM (Document Object Model) représente la structure hiérarchique des éléments HTML sur une page web.

Cette arborescence peut être considérée comme un système parent-enfant, dans lequel chaque élément est un enfant d'un autre élément. Par exemple, le body est l'enfant direct de l'élément html.

Pour ajouter des éléments à une liste de nœuds, vous pouvez utiliser les fonctions appendChild() ou insertBefore().
La méthode appendChild() ajoute un nouvel élément en tant que dernier enfant d'un élément spécifié.

Voici un exemple :

```
const parent = document.getElementById('parent');
const newElement = document.createElement('li');
newElement.textContent = 'Nouvel élément ajouté';
parent.appendChild(newElement);
```

Dans cet exemple, nous avons d'abord récupéré l'élément parent avec la méthode getElementById().

Nous avons ensuite créé un nouvel élément li avec la méthode createElement() et nous avons défini son texte avec la propriété textContent. Enfin, nous avons ajouté le nouvel élément en tant que dernier enfant de l'élément parent en utilisant la méthode appendChild().

La méthode insertBefore() ajoute un nouvel élément avant un élément spécifié dans la liste des enfants d'un autre élément. Voici un exemple :

```
const parent = document.getElementById('parent');
const newElement = document.createElement('li');
newElement.textContent = 'Nouvel élément ajouté';
const refElement = parent.firstChild;
parent.insertBefore(newElement, refElement);
```

Dans cet exemple, nous avons d'abord récupéré l'élément parent avec la méthode getElementById().

Nous avons ensuite créé un nouvel élément li avec la méthode createElement() et nous avons défini son texte avec la propriété textContent.

Nous avons ensuite récupéré le premier enfant de l'élément parent avec la propriété firstChild. Enfin, nous avons ajouté le nouvel élément avant le premier enfant de l'élément parent en utilisant la méthode insertBefore().

Il est important de noter que ces méthodes ajoutent les éléments à la fin ou au début de la liste des enfants d'un élément spécifié. Si vous souhaitez ajouter un élément à un endroit spécifique dans la liste, vous devrez d'abord récupérer l'élément avant lequel vous souhaitez ajouter le nouvel élément, comme dans l'exemple ci-dessus.

LA MINUTE DE CODE

LA BOUCLE FOR

La boucle for est une structure de contrôle de flux en JavaScript qui permet de répéter une série d'instructions un certain nombre de fois.

Cette boucle est particulièrement utile lorsque vous devez effectuer une opération un nombre connu de fois.

La syntaxe générale de la boucle for en JavaScript est la suivante :

```
for (initialisation; condition; incrémentation) {
  // Instructions à répéter
}
```

Les trois parties principales de la boucle for sont les suivantes :

- **L'initialisation :** il s'agit d'une instruction qui est exécutée une seule fois avant le début de la boucle. Elle est souvent utilisée pour définir une variable qui sera utilisée pour compter le nombre de tours de boucle. Par exemple, let i = 0; initialise une variable i à 0.
- **La condition :** il s'agit d'une expression qui est évaluée avant chaque tour de boucle. Si l'expression est vraie, la boucle continue ; sinon, elle s'arrête. Par exemple, i < 10 teste si la variable i est inférieure à 10.
- **L'incrémentation :** Une instruction qui est exécutée à la fin de chaque itération, souvent pour modifier la valeur de la variable initialisée.

Voici un exemple de boucle for qui itère sur un tableau et affiche chaque élément dans la console :

```
const tableau = ['un', 'deux', 'trois'];

for (let i = 0; i < tableau.length; i++) {
  console.log(tableau[i]);
}
```

LA MINUTE DE CODE

LA BOUCLE FOR

Dans cet exemple, la boucle for commence par initialiser une variable i à 0. La condition de la boucle vérifie si i est inférieur à la longueur du tableau tableau. Si c'est le cas, les instructions à l'intérieur de la boucle sont exécutées, dans ce cas l'affichage de l'élément du tableau correspondant à l'index i.

Ensuite, la variable i est incrémentée de 1 à chaque itération de la boucle.

La boucle for est une structure de contrôle de flux très utile en JavaScript, qui permet de parcourir des tableaux, des objets et d'autres types de collections d'éléments, ainsi que d'exécuter une série d'instructions un certain nombre de fois.

LA MINUTE DE CODE

WHILE ET DO WHILE

JS

La boucle while et la boucle do-while sont deux structures de contrôle de flux en JavaScript qui permettent de répéter une série d'instructions tant qu'une condition est vraie.

La principale différence entre les deux est que la boucle do-while garantit l'exécution de l'instruction au moins une fois, tandis que la boucle while peut ne jamais être exécutée si la condition est fausse dès le début.

La syntaxe de la boucle while est la suivante :

```
while (condition) {
  // instructions à exécuter tant que
la condition est vraie
}
```

La boucle while commence par évaluer la condition.

Si la condition est vraie, les instructions à l'intérieur de la boucle sont exécutées. Ensuite, la condition est évaluée à nouveau, et ainsi de suite jusqu'à ce que la condition devienne fausse. Si la condition est fausse dès le début, les instructions à l'intérieur de la boucle ne sont jamais exécutées.

Voici un exemple de boucle while qui affiche des nombres de 1 à 5 dans la console :

```
let i = 1;

while (i <= 5) {
  console.log(i);
  i++;
}
```

WHILE ET DO WHILE

Dans cet exemple, la boucle while commence par initialiser une variable i à 1. La condition de la boucle vérifie si i est inférieur ou égal à 5.

Tant que la condition est vraie, les instructions à l'intérieur de la boucle sont exécutées, dans ce cas l'affichage de la valeur de i et l'incrémentation de i de 1 à chaque itération.

La syntaxe de la boucle do-while est la suivante :

```
do {
  // instructions à exécuter tant que
la condition est vraie
} while (condition);
```

La boucle do-while commence par exécuter les instructions à l'intérieur de la boucle une fois, sans évaluer la condition. Ensuite, la condition est évaluée.

Si la condition est vraie, les instructions à l'intérieur de la boucle sont exécutées à nouveau, et ainsi de suite jusqu'à ce que la condition devienne fausse.

Voici un exemple de boucle do-while qui affiche des nombres de 1 à 5 dans la console :

```
let i = 1;

do {
  console.log(i);
  i++;
} while (i <= 5);
```

Dans cet exemple, la boucle do-while commence par exécuter les instructions à l'intérieur de la boucle une fois, sans évaluer la condition.

Ensuite, la condition de la boucle vérifie si i est inférieur ou égal à 5. Tant que la condition est vraie, les instructions à l'intérieur de la boucle sont exécutées, dans ce cas l'affichage de la valeur de i et l'incrémentation de i de 1 à chaque itération.

En résumé, la boucle while et la boucle do-while sont deux structures de contrôle de flux en JavaScript qui permettent de répéter une série d'instructions tant qu'une condition est vraie. La différence principale entre les deux est que la boucle `

LA MINUTE DE CODE

LES TABLEAUX

JS

En JavaScript, les tableaux sont des structures de données qui permettent de stocker des collections de valeurs. Chaque valeur dans un tableau est associée à un index numérique, qui commence toujours à zéro.

Les tableaux peuvent contenir des valeurs de différents types, y compris des nombres, des chaînes de caractères, des objets et même d'autres tableaux.

Les tableaux en JavaScript sont dynamiques, ce qui signifie que leur taille peut être modifiée dynamiquement pendant l'exécution du programme. Cela signifie que vous pouvez ajouter et supprimer des éléments du tableau à tout moment.

Pour accéder à une valeur dans un tableau, vous pouvez utiliser l'index numérique correspondant à cette valeur. Par exemple, si vous avez un tableau contenant trois éléments, vous pouvez accéder au deuxième élément en utilisant l'index 1, comme ceci :

```
let myArray = [10, "hello", true];
let secondElement = myArray[1]; // "hello"
```

Dans cet exemple, nous avons créé un tableau myArray qui contient trois éléments de types différents. Nous avons ensuite utilisé l'index 1 pour accéder au deuxième élément du tableau, qui est une chaîne de caractères contenant le texte "hello".

Il est important de noter que les index de tableau commencent toujours à zéro en JavaScript. Ainsi, le premier élément d'un tableau a un index de 0, le deuxième élément a un index de 1, et ainsi de suite.

LES TABLEAUX

Les tableaux peuvent également être utilisés pour stocker des collections d'objets. Par exemple, vous pouvez avoir un tableau d'objets qui représentent des personnes, comme ceci :

```
let people = [
  { name: "John", age: 25 },
  { name: "Mary", age: 30 },
  { name: "Bob", age: 40 }
];
```

Dans cet exemple, nous avons créé un tableau people qui contient trois objets, chacun représentant une personne avec un nom et un âge.

Nous pouvons accéder à chaque objet dans le tableau en utilisant son index numérique.

En conclusion, les tableaux en JavaScript sont des structures de données utiles qui permettent de stocker des collections de valeurs, y compris des nombres, des chaînes de caractères, des objets et d'autres tableaux. Les index de tableau commencent à zéro, et les tableaux sont dynamiques, ce qui signifie qu'ils peuvent être modifiés à tout moment.

METHODES LIÉES AUX TABLEAUX

La méthode length

En JavaScript, la méthode .length est utilisée pour retourner la longueur d'un tableau ou d'une chaîne de caractères.

Pour un tableau, la méthode .length retourne le nombre d'éléments dans le tableau. Par exemple, si vous avez un tableau de 5 éléments, la méthode .length retournera la valeur 5. Voici un exemple :

```
let myArray = [10, "hello", true, null, undefined];
console.log(myArray.length); // 5
```

Dans cet exemple, nous avons créé un tableau myArray qui contient cinq éléments de différents types.

Nous avons ensuite utilisé la méthode .length pour afficher le nombre d'éléments dans le tableau, qui est 5.

Pour une chaîne de caractères, la méthode .length retourne le nombre de caractères dans la chaîne. Par exemple, si vous avez une chaîne de caractères contenant 10 caractères, la méthode .length retournera la valeur 10. Voici un exemple :

```
let myString = "Hello World";
console.log(myString.length); // 11
```

Dans cet exemple, nous avons créé une chaîne de caractères myString qui contient 11 caractères.

Nous avons ensuite utilisé la méthode .length pour afficher le nombre de caractères dans la chaîne, qui est 11.

LA MINUTE DE CODE

METHODES LIÉES AUX TABLEAUX

Les méthodes "pop", "push", "unshift" et "shift" sont des méthodes utilisées pour manipuler des tableaux en JavaScript.

La méthode push

La méthode "push" ajoute un ou plusieurs éléments à la fin du tableau. Voici un exexemple

```
let fruits = ["pomme", "banane"];
fruits.push("orange");
console.log(fruits); // ["pomme", "banane", "orange"]
```

La méthode pop

La méthode "pop" supprime le dernier élément du tableau et renvoie cet élément. Voici un exemple:

```
let fruits = ["pomme", "banane", "orange"];
let dernierFruit = fruits.pop();
console.log(dernierFruit); // "orange"
console.log(fruits); // ["pomme", "banane"]
```

La méthode unshift

La méthode "unshift" ajoute un ou plusieurs éléments au début du tableau. Voici un exemple:

```
let fruits = ["pomme", "banane"];
fruits.unshift("orange");
console.log(fruits); // ["orange", "pomme", "banane"]
```

METHODES LIÉES AUX TABLEAUX

La méthode shift

La méthode "shift" supprime le premier élément du tableau et renvoie cet élément. Voici un exemple:

```
let fruits = ["orange", "pomme", "banane"];
let premierFruit = fruits.shift();
console.log(premierFruit); // "orange"
console.log(fruits); // ["pomme", "banane"]
```

La méthode splice

La méthode "splice" est utilisée pour ajouter ou supprimer des éléments à l'intérieur du tableau. La méthode prend en entrée un index de départ, le nombre d'éléments à supprimer (facultatif) et les éléments à ajouter (facultatif). Voici un exemple:

```
let fruits = ["pomme", "banane", "orange"];
fruits.splice(1, 1, "kiwi", "ananas");
console.log(fruits); // ["pomme", "kiwi", "ananas",
"orange"]
```

METHODES LIÉES AUX TABLEAUX

La méthode slice

La méthode "slice" est utilisée pour créer une nouvelle copie d'une partie d'un tableau. La méthode prend en entrée un index de départ et un index de fin (facultatif). Voici un exemple:

```
let fruits = ["pomme", "banane", "orange", "kiwi",
"ananas"];
let sousTableau = fruits.slice(1, 4);
console.log(sousTableau); // ["banane", "orange",
"kiwi"]
```

La méthode concat

La méthode "concat" est utilisée pour fusionner deux tableaux ou plus en un seul tableau. Voici un exemple :

```
let fruits = ["pomme", "banane"];
let legumes = ["carotte", "pomme de terre"];
let produits = fruits.concat(legumes);
console.log(produits); // ["pomme", "banane",
"carotte", "pomme de terre"]
```

Dans l'exemple ci-dessus, nous avons fusionné les tableaux "fruits" et "légumes" en un seul tableau appelé "produits".

METHODES LIÉES AUX TABLEAUX

La méthode flat

La méthode "flat" est utilisée pour aplatir un tableau en retirant les tableaux imbriqués. Voici un exemple :

```
let tableauxImbriques = [1, [2, 3], [4, [5, 6]]];
let tableauAplatit = tableauxImbriques.flat();
console.log(tableauAplatit); // [1, 2, 3, 4, 5, 6]
```

Dans l'exemple ci-dessus, nous avons utilisé la méthode "flat" pour aplatir le tableau "tableauxImbriques".

La méthode join

La méthode "join" est utilisée pour convertir un tableau en une chaîne de caractères en joignant tous les éléments du tableau avec un séparateur. Voici un exemple :

```
let fruits = ["pomme", "banane", "orange"];
let chaineDeCaracteres = fruits.join(", ");
console.log(chaineDeCaracteres); // "pomme, banane,
orange"
```

Dans l'exemple ci-dessus, nous avons utilisé la méthode "join" pour convertir le tableau "fruits" en une chaîne de caractères en les séparant par une virgule et un espace.

LA MINUTE DE CODE

METHODES LIÉES AUX TABLEAUX

JS

La méthode forEach

La méthode "forEach" est utilisée pour exécuter une fonction sur chaque élément d'un tableau. Voici un exemple :

```
let nombres = [1, 2, 3];
nombres.forEach(function(nombre) {
  console.log(nombre);
});
// Affiche 1
// Affiche 2
// Affiche 3
```

Dans l'exemple ci-dessus, nous avons utilisé la méthode "forEach" pour afficher chaque élément du tableau "nombres".

La méthode map

La méthode "map" est utilisée pour créer un nouveau tableau en appliquant une fonction à chaque élément du tableau d'origine. Voici un exemple :

```
let nombres = [1, 2, 3];
let doubles = nombres.map(function(nombre) {
  return nombre * 2;
});
console.log(doubles); // [2, 4, 6]
```

Dans l'exemple ci-dessus, nous avons utilisé la méthode "map" pour créer un nouveau tableau appelé "doubles" qui contient chaque élément du tableau "nombres" multiplié par 2.

METHODES LIÉES AUX TABLEAUX

La méthode filter

La méthode "filter" est utilisée pour créer un nouveau tableau contenant tous les éléments qui satisfont une condition donnée. Voici un exemple :

```
let nombres = [1, 2, 3, 4, 5];
let nombresPairs = nombres.filter(function(nombre) {
  return nombre % 2 === 0;
});
console.log(nombresPairs); // [2, 4]
```

Dans l'exemple ci-dessus, nous avons utilisé la méthode "filter" pour créer un nouveau tableau appelé "nombresPairs" qui contient tous les nombres pairs du tableau "nombres".

La méthode reduce

La méthode "reduce" est utilisée pour réduire tous les éléments d'un tableau à une seule valeur en appliquant une fonction sur chaque élément. Voici un exemple :

```
let nombres = [1, 2, 3, 4, 5];
let somme = nombres.reduce(function(total, nombre) {
  return total + nombre;
}, 0);
console.log(somme); // 15
```

Dans l'exemple ci-dessus, nous avons utilisé la méthode "reduce" pour calculer la somme de tous les éléments du tableau "nombres".

METHODES LIÉES AUX TABLEAUX

La méthode every

La méthode "every" est utilisée pour vérifier si tous les éléments d'un tableau satisfont une condition donnée. Voici un exemple :

```
let nombres = [2, 4, 6, 8];
let tousPairs = nombres.every(function(nombre) {
  return nombre % 2 === 0;
});
console.log(tousPairs); // true
```

Dans l'exemple ci-dessus, nous avons utilisé la méthode "every" pour vérifier si tous les nombres du tableau "nombres" sont pairs.

La méthode fill

La méthode "fill" est utilisée pour remplir tous les éléments d'un tableau avec une valeur donnée. Voici un exemple :

```
let nombres = [1, 2, 3, 4, 5];
nombres.fill(0);
console.log(nombres); // [0, 0, 0, 0, 0]
```

Dans l'exemple ci-dessus, nous avons utilisé la méthode "fill" pour remplir tous les éléments du tableau "nombres" avec la valeur 0.

METHODES LIÉES AUX TABLEAUX

La méthode copywithin

La méthode "copyWithin" est utilisée pour copier une plage d'éléments d'un tableau dans une autre plage d'éléments du même tableau. Voici un exemple :

```
let nombres = [1, 2, 3, 4, 5];
nombres.copyWithin(0, 3, 5);
console.log(nombres); // [4, 5, 3, 4, 5]
```

Dans l'exemple ci-dessus, nous avons utilisé la méthode "copyWithin" pour copier les éléments du tableau "nombres" situés entre les index 3 et 4 dans la plage d'éléments située entre les index 0 et 2.

La méthode entries

La méthode "entries" est utilisée pour créer un nouvel objet itérable qui contient des paires clé-valeur pour chaque élément du tableau. Voici un exemple :

```
let fruits = ['pomme', 'banane', 'cerise'];
let entrees = fruits.entries();
for (let [index, fruit] of entrees) {
  console.log(index, fruit);
}
// 0 "pomme"
// 1 "banane"
// 2 "cerise"
```

Dans l'exemple ci-dessus, nous avons utilisé la méthode "entries" pour créer un nouvel objet itérable "entrees" contenant des paires clé-valeur pour chaque élément du tableau "fruits". Nous avons ensuite utilisé une boucle for...of pour parcourir chaque élément de l'objet itérable et afficher son index et sa valeur.

METHODES LIÉES AUX TABLEAUX

La méthode sort

La méthode "sort" est utilisée pour trier les éléments d'un tableau selon un ordre donné. Voici un exemple :

```
let nombres = [2, 5, 1, 3, 4];
nombres.sort();
console.log(nombres); // [1, 2, 3, 4, 5]
```

Dans l'exemple ci-dessus, nous avons utilisé la méthode "sort" pour trier les éléments du tableau "nombres" en ordre croissant. Il est important de noter que la méthode "sort" trie le tableau directement et ne crée pas de nouveau tableau. De plus, par défaut, la méthode "sort" trie les éléments du tableau comme s'ils étaient des chaînes de caractères. Pour trier des nombres dans l'ordre croissant, il faut utiliser une fonction de comparaison comme ceci :

```
let nombres = [2, 5, 1, 3, 4];
nombres.sort(function(a, b) {
  return a - b;
});
console.log(nombres); // [1, 2, 3, 4, 5]
```

Dans l'exemple ci-dessus, nous avons utilisé la méthode "sort" avec une fonction de comparaison qui trie les nombres dans l'ordre croissant. La fonction de comparaison compare deux nombres en soustrayant le premier du deuxième. Si le résultat est négatif, cela signifie que le premier nombre est inférieur au deuxième et doit être placé avant lui dans le tableau trié. Si le résultat est positif, cela signifie que le premier nombre est supérieur au deuxième et doit être placé après lui dans le tableau trié. Si le résultat est zéro, cela signifie que les deux nombres sont égaux et leur ordre ne change pas.

LA BOUCLE FOR OF

La boucle "for...of" est une boucle JavaScript qui permet de parcourir les éléments d'un objet itérable, tels que les tableaux, les chaînes de caractères et les objets de type "Map" et "Set", de manière simple et lisible.

La syntaxe de la boucle "for...of" est la suivante :

```
for (variable of iterable) {
  // instructions à exécuter pour chaque élément de l'itérable
```

La variable est une variable qui prend la valeur de chaque élément de l'itérable à chaque itération de la boucle. L'itérable est l'objet que l'on souhaite parcourir. À chaque itération de la boucle, la variable prend la valeur de l'élément suivant dans l'itérable.
Voici un exemple de boucle "for...of" qui parcourt les éléments d'un tableau :

```
let fruits = ['pomme', 'banane', 'cerise'];
for (let fruit of fruits) {
  console.log(fruit);
}
// "pomme" // "banane" // "cerise"
```

Dans l'exemple ci-dessus, nous avons utilisé la boucle "for...of" pour parcourir chaque élément du tableau "fruits" et afficher sa valeur à chaque itération.

La boucle "for...of" est pratique car elle simplifie la syntaxe de la boucle "for" traditionnelle et évite d'avoir à utiliser des index ou des fonctions de boucle. Elle est également plus lisible et plus facile à comprendre. Cependant, il est important de noter que la boucle "for...of" ne fournit pas l'index de chaque élément dans l'itérable, contrairement à la boucle "for" traditionnelle.

LA MINUTE DE CODE

LA MÉTHODE VALUE

JS

La méthode "values()" renvoie un objet itérateur qui parcourt tous les éléments du tableau dans l'ordre de leur index, en ignorant les clés des propriétés définies dans le tableau. Vous pouvez utiliser cette méthode pour obtenir une liste des valeurs des éléments du tableau, sans leurs indices.

Voici un exemple d'utilisation de la méthode "values()" pour obtenir une liste des valeurs des éléments d'un tableau :

```
const array = [1, 2, 3];
const values = array.values();

for (const value of values) {
  console.log(value);
}
// Output: 1
// Output: 2
// Output: 3
```

Dans l'exemple ci-dessus, nous avons créé un tableau "array" contenant des valeurs numériques et avons créé un itérateur "values" en appelant la méthode "values()" sur le tableau. Ensuite, nous avons utilisé une boucle "for...of" pour parcourir chaque élément de l'itérateur "values" et afficher sa valeur dans la console.

Notez que la méthode "values()" n'est pas prise en charge dans certains navigateurs plus anciens, tels qu'Internet Explorer, alors assurez-vous de vérifier la compatibilité du navigateur avant de l'utiliser.



laminutedecode.io

@LAMINUTEDECODE

